Geometry Reveal Causality：

Detecting Reasoning Hallucination in Long-Chain LLM Reasoning via Multi-Component Contextual Anchor in Embedding Space and Machine Learning

Name：Yu Ruotong

Abstract

Large Language Models (LLMs) often produce hallucinations during long-chain causal reasoning. In tasks like policy impact analysis, the model must reason through multiple steps, and small errors at each step accumulate over time, causing the final conclusion to drift far from the original topic. Instead of directly checking reference from the internet, we bring out an logic-based method to detect LLM hallucination.

In this paper, we present three key contributions to hallucination detection in large language models (LLMs). First, we introduce a novel detection framework grounded in the geometric structure of embedding space. We discover that LLM hallucinations exhibit identifiable geometric patterns when reasoning steps are encoded as high-dimensional vectors. Building on this insight, we employ a BGE (BAAI General Embedding) model to map reasoning steps into 1024-dimensional vectors and design a central multi-component vector that captures the main thesis of a reasoning chain, serving as a context-aware baseline. A lightweight Multi-Layer Perceptron (MLP) classifier—with only 2.2M parameters—is then trained to learn these geometric patterns and distinguish hallucinated steps from logically sound ones. Second, we contribute a curated dataset of 4,293 reasoning samples across 10 policy themes, constructed through an agent-based data pipeline with LLM-as-a-Judge quality control. On this benchmark, our best model (V2, trained with balanced positive and negative samples) achieves 91.63% accuracy and 89.13% Macro-F1, significantly outperforming baselines including cosine similarity, static anchor ablation, and zero-shot LLM classification. Third, we provide a critical real-world evaluation by deploying the model on an unseen policy theme, where it reveals notable limitations in detecting shifts in

abstraction level and in overall generalization capacity. We analyze the underlying causes of these failures and outline promising directions for future research.

Keywords: LLM hallucination, embedding space geometry, probing classifiers, policy analysis, NLP for sociology, semantic learning

1 Introduction

1.1 Research Background: LLM Hallucination and Long-Chain Reasoning Challenges

Large Language Models have shown strong abilities in text generation and reasoning, but they also produce hallucinations—outputs that look fluent and confident but are factually wrong or logically unsound. Huang et al. (2023) and Ji et al. (2023) provide surveys of this problem and group LLM hallucinations into three types: factual hallucination, where the output contains wrong facts; faithfulness hallucination, where the output does not follow the given source or instruction; and reasoning hallucination, where the output contains flaws in the logical chain, such as false cause-effect links. Among these, reasoning hallucination is the hardest to detect because the output may look reasonable on the surface while the underlying logic is broken.

This problem becomes much more serious in long-chain reasoning tasks. In fields like policy analysis, risk assessment, and supply chain tracing, LLMs need to perform multi-hop causal reasoning, for example from a policy to its first-order impact, then to a second-order impact, and further to a third-order impact. Each step depends on the one before it. OpenAI (2025) points out that standard training procedures tend to reward guessing rather than expressing uncertainty. In a long auto-regressive chain, this tendency creates an error snowballing effect: a small mistake at an early step gets amplified and passed along to later steps, making the whole chain gradually drift away from the truth.

We define this problem as progressive semantic drift. At each reasoning step, there may be a small local deviation in the semantic space. These deviations accumulate over multiple hops, and as a result, the overall reasoning path slowly moves away from the original policy context. The final output may still read smoothly and appear to make sense, but the causal relationship it describes does not actually hold. This is a invalid-causal relationship — a hallucination that is hard to catch by reading alone.

The issue is especially crucial for policy analysis. Policy impact researchers often use LLM as a tool to summarize the corpus and study how the impact of the policy pass through different social groups. This effort is helpful for making proper policy that has maximized benefit scope. The reasoning hallucination might cause mistakes while LLM is studying the social relations and impact causality. The illusion will pass to the policy itself, which will finally lead to the inequality in rights and resources distribution.

For example, in an analysis of an policy about steel production, hallucination might regard relation between “steel factories” and “rural health” as “no relation”, and not provide information about the issue. Thus the policy maker will probably ignore the health issue in villages caused by factories’ pollution, leading to inequality and ignorance.

This oversight and misunderstanding to the logic frequently happen, which it appears when the detection method mostly rely on checking the factual evidence but forget to check the broken logical rationality. For now, hallucination detecting methods are mostly focusing on fact-based methods. This kind of methods mainly detect factual hallucination, leaving reasoning hallucination undetected.

1.2 Limitations of Existing Methods

Researchers have proposed several methods to detect LLM hallucinations, but each has clear limitations. The LLM-as-a-Judge approach proposed by Zheng et al. (2023) is flexible and interpretable, yet it comes with high compute cost, high latency, and inconsistent scoring across different runs. Traditional knowledge graph verification methods can check factual accuracy, but they have limited coverage, cannot handle new concepts, and are not suitable for causal reasoning tasks. The self-consistency detection method of Wang et al. (2023) does not require external knowledge, but it needs multiple sampling rounds, which increases cost, and it is ineffective against systematic bias that affects all samples in the same way.

The core gap shared by all these methods is that they treat hallucination detection as a text-level semantic matching problem. They compare the surface meaning of texts or rely on external knowledge bases. However, they overlook a valuable source of information: the rich geometric structure inside the model's own embedding space. When an LLM processes text, it maps each piece of text into a high-dimensional vector, and these vectors carry information about meaning, direction, and logical structure. We believe this geometric information can be used to detect whether a causal reasoning step is valid or spurious.

1.3 Research Hypothesis and Core Idea

Our core hypothesis is that the validity of causal reasoning can be judged through vector geometric patterns in the embedding space. Genuine causal reasoning should form a coherent, directional trajectory in the semantic space that follows certain geometric constraints. In contrast, spurious causal reasoning, which is a form of hallucination, should show incoherent, off-direction, or otherwise abnormal geometric features.

Based on this hypothesis, we shift the hallucination detection problem from text semantic matching to high-dimensional geometric pattern recognition. Instead of comparing the surface text of two entities, we look at where they sit in the embedding space, how they move from one to the next, and whether that movement is consistent with the overall direction of the reasoning chain. Our method follows the probing classifier paradigm described by Belinkov and Glass (2019), where the pre-trained model is kept frozen, its internal representations are extracted, and a lightweight network is trained to detect specific properties. In our case, we use the BGE embedding model to generate vector representations, design four geometric features from these vectors, and train a small MLP to classify each reasoning step as valid or spurious.

1.4 Main Contributions

In this paper, we attempt to use geometric patterns in embedding space to explain logical patterns,and we train a MLP model to detect reasoning hallucination by solely observing the logic.

We propose a Multi-Agent Data Elicitation pipeline combined with LLM-as-a-Judge to clean the dataset. This data flywheel produces 4,293 annotated samples across 10 diverse policy themes, covering agriculture, education, technology, healthcare, environment, and social security, providing a broad empirical foundation for the study.

At the methodology level, we propose a Multi-component Central Vector that combines the policy topic vector with the first two reasoning steps. This design provides a new way giving context to the model (as well as attention and memory). Combined with four geometric features and a lightweight MLP probe, the model achieves 91.63% accuracy and 89.13% Macro-F1 on the benchmark test set, while the ablation study confirms that the multi-component centroid contributes a 7.1% gain in Macro-F1 over a static anchor.

In our experiments, our results partially support the hypothesis that causal drift has geometric signatures in the embedding space, because the geometric features alone are sufficient to achieve strong benchmark performance. However, when deployed on an unseen policy theme in a real-world test, the model's performance is notably weaker. It struggles with cross-level abstraction transitions and unseen policy domains. These findings reveal that current geometric features and model capacity are not yet sufficient for robust real-world deployment, and we provide a detailed discussion of these limitations along with suggestions for future work.

1.5 Guide for Readers

In this paper, we assume that the reader has basic knowledge on Large Language Model’s hallucination, common evaluation metrics and mathematics, we will explain all the purpose and reason of the setting, but we won’t explain the basic operating principle in maths or computer science. For the part of embedding method and feature engineering, we will give sufficient introduce on it to help readers understand our new method and the limitations.

2 Related Work

2 Related Work- Embedding Space Analysis and Probing Methods

If we want to figure out what is actually going on inside an LLM, we usually have to dig into its embedding space. Word embedding is a way we use vectors to describe the semantic, or the meaning , of the text, using mathematics to analyse patterns in text. The standard trick here is the probing classifier: you freeze the pre-trained model, pull out its hidden representations, and train a simple classifier to see if certain linguistic or semantic features are actually encoded in there (Belinkov & Glass, 2019). It is a well-established paradigm, but applying it to raw geometry is trickier than it sounds.

The main headache is that contextualized word embeddings are highly anisotropic. Ethayarajh (2019) showed that instead of spreading out evenly, word vectors tend to get squeezed into a narrow cone. If you just blindly measure distances in this space, the results get heavily distorted. On top of that, you have the "rogue dimensions" problem described by Timkey and van Schijndel (2021) — a handful of specific dimensions that blow up to disproportionately large values and end up dominating the distance metrics, masking the actual representational quality.

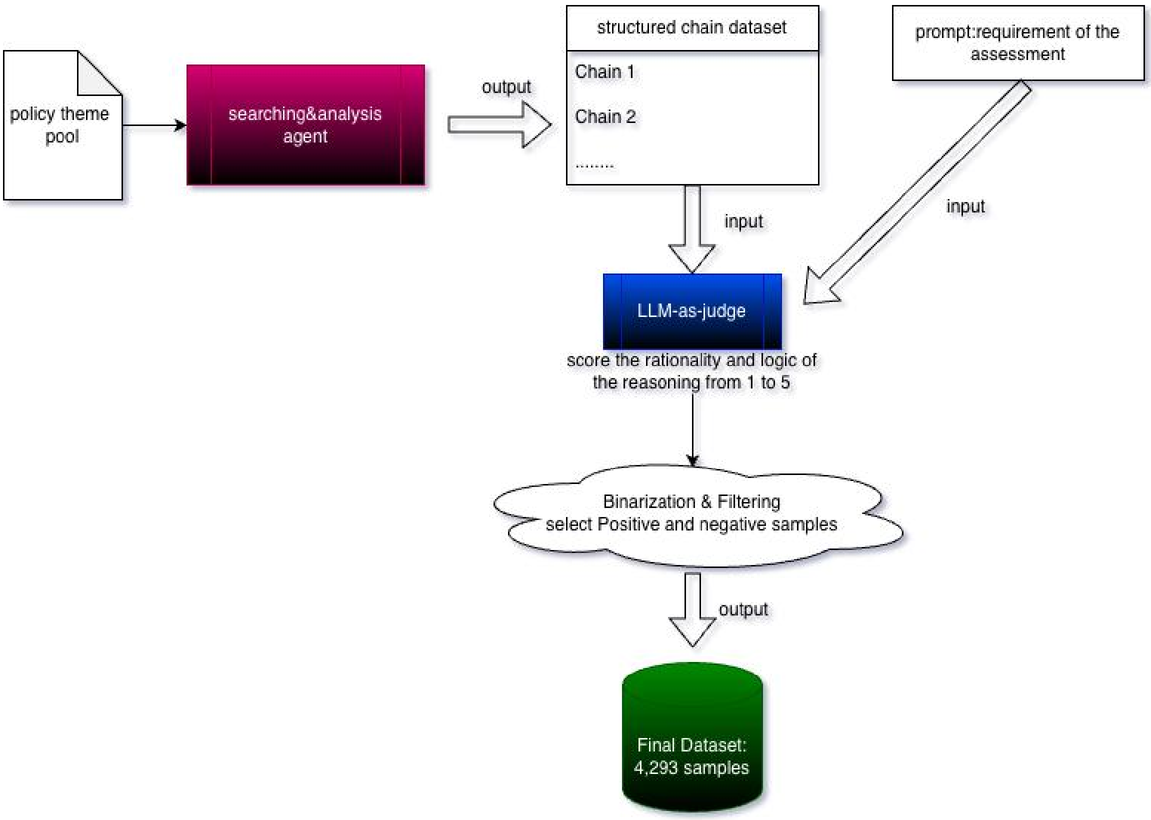
These geometric quirks are not just theoretical footnotes. If we ignore them, any distance-based analysis or feature engineering we do on the embeddings will be fundamentally flawed. Recognizing and working around these issues forms the practical basis for the probing methods we designed in this study.

3 Data Construction Pipeline

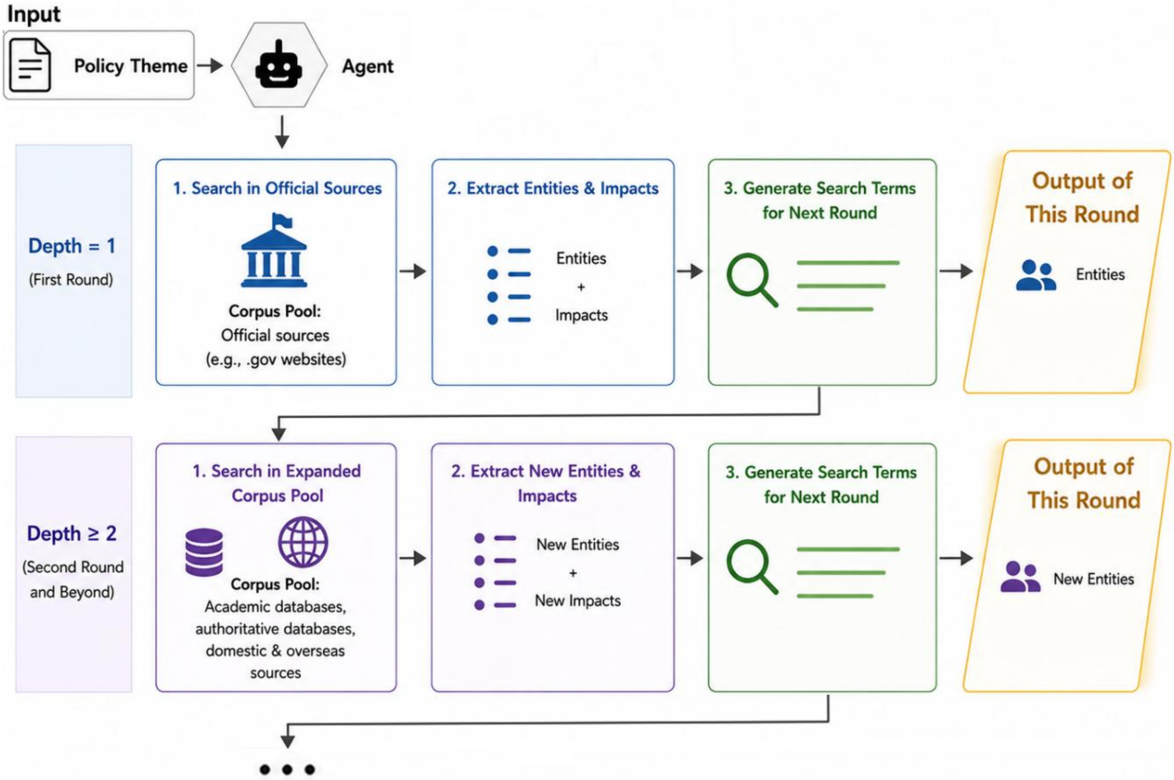
3.1 Agent Data Elicitation with LLM-as-a-Judge

3.1.1 Agent Design

In order to produce sufficient data, we have designed an end-to-end automated data pipeline based on LLM agent and LLM-as-Judge method, which simulates real logical reasoning scenarios, ensuring the structural integrity of causal chains and providing high-quality data for model training.



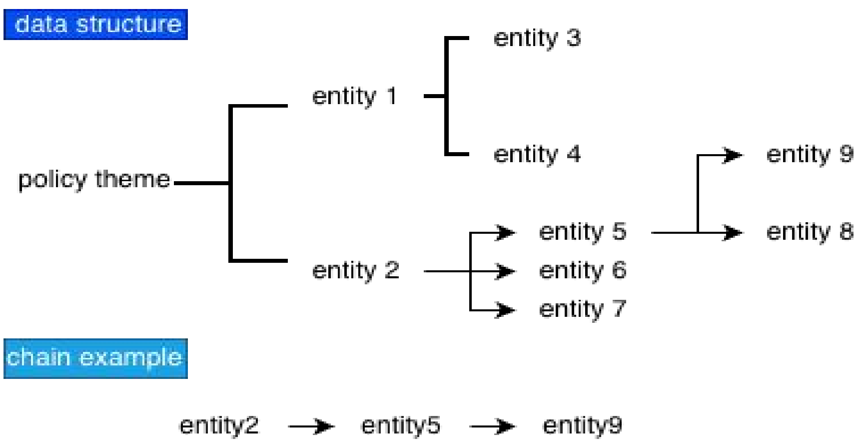
Through multi-hop search and concurrent structure, the agent can identify the social groups that are directly or indirectly affected by the policy, and generate a tree-like data structure. We can separate this data structure and form a large quantity of chain-like data structure, which is called the **reasoning chain.** The social group that been searched is called **entities**.



Given a policy theme, the agent conducts iterative literature searches. At depth 1, it queries official sources (e.g., .gov sites) to identify entities directly affected by the policy and their corresponding impacts, which the LLM uses to generate refined search terms. At depth ≥ 2, the search scope expands to domestic and international academic and authoritative databases. In each subsequent round, the LLM extracts new entity-impact pairs from the retrieved documents to dynamically generate search terms for the next iteration.

3.1.2 Structured Reasoning Data

A new discovery of entity is directly attribute to the latest discovery of entity. This mechanism creates a search loop, enabling the agent to generate reasoning chains with high depth, thereby simulating the semantic drift that occurs during policy impact analysis. The longer the chain, the more indirect the impact will be, and the more likely semantic drift will occur. Entities unrelated to the actual policy can be identified within the impact chain.



The graph illustrates the output data structure, where tree-like data is decomposed into chains. Using a tailored prompt and JSON Schema constraints, the LLM is forced to produce hierarchically structured causal chains from Depth 0 to Depth 3. Each chain contains: (1) policy — the initial policy or event; (2) depth_1 — first-order impacts on 2–3 entities; (3) depth_2 — second-order impacts, with each depth_1 entity branching into 2–3 more; (4) depth_3 — third-order impacts, with each depth_2 entity extending to 1–2 further entities; and (5) reasoning — a brief justification for each inference step. This structure ensures logical organization, traceability, and interpretability throughout the causal chain.

In this research, we use single step of the reasoning as training samples. In the example, we can extract two steps of reasoning, which is the two samples:

chain example

entity2 → entity5 → entity9

cut into——

entity2 → entity5

entity5 → entity9

We cannot guarantee that different policies will have the same degree of impact and the same number of affected social groups in all cases. So we will cut the chain into single-step reasoning units and use them as samples. This approach enables the model to focus on single-step reasoning, avoids excessive feature quantities, and solves the problem of inconsistent chain lengths. It also allows for the most comprehensive use of the data.

In the research, we generate 1292 chains, and gain 5742 initial samples from it. After the drop, 4293 samples left.

3.2 Choice of Policy Theme

To ensure that our model can work well across different fields, we carefully selected 10 different policy topics.

No.	Policy Topic	Domain / Category
1	Abolition of Agricultural Tax	Agriculture / Finance

No.	Policy Topic	Domain / Category
2	Anti-addiction Rules for Video Games	Entertainment / Regulation
3	Relaxation of Housing Purchase Restrictions	Real Estate / Finance
4	Subsidies for New Energy Vehicles	Industry / Environment
5	"Double Reduction" Education Policy	Education
6	Carbon Emission Trading	Environment / Energy
7	Centralized Procurement of Medical Supplies	Healthcare
8	Anti-monopoly Regulation of Tech Platforms	Technology / Law
9	Tax Incentives for Technological Innovation	Technology / Finance
10	Social Security System Reform	Social Security

We selected 10 diverse policies spanning the economy, environment, education, technology, and public welfare to demonstrate the dataset's comprehensiveness. These policies encompass various government actions—financial incentives, behavioral restrictions, and institutional reforms—differing widely in goals, mechanisms, and affected populations. This diversity ensures our method is validated as a generalizable tool applicable to a broad range of real-world policy scenarios.

3.3 LLM-as-a-Judge Quality Control

The causal edges generated in Phase I may contain noise or ambiguous quality of reasoning. These sample is vague to judge whether the reasoning is valid, which means that these kind of data can’t be used for training. To ensure the reliability of the training data for the geometric probe, we introduce an automated quality control mechanism using a LLM as an independent judge. This phase systematically evaluates each causal pair, filters out low-quality or ambiguous samples, and constructs a high-confidence binary classification dataset.

3.3.1 Judge Design

We employ Deepseek V4 Pro as the LLM judge to serve as an automated evaluator. For each generated causal pair $(\text{parent}, \text{child})$, the Judge assesses the relationship from three distinct dimensions:

- Logical Plausibility: Does the parent event reasonably lead to the child event? This dimension examines whether the causal link is logically sound and supported by real-world reasoning.
- Causal Directionality: Is the direction of causality correct? The Judge verifies that the relationship represents a true causal effect (parent → child) rather than a mere correlation, reverse causation, or coincidental association.
- Contextual Consistency: Does the causal relationship remain within the scope of the original policy context? This ensures that the generated edges do not drift into irrelevant or overly speculative domains.

Each dimension is scored on a scale from 1 to 5, and the Judge provides a final comprehensive score based on the overall assessment across all three dimensions.

3.3.2 Scoring Rubric

The Judge assigns an overall score from 1 to 5 for each causal pair. Table 1 presents the detailed scoring criteria and the corresponding processing decision for each score level.

Scoring Rubric and Processing Strategy：

Score	Meaning	Description	Processing
5	Strong Causality	Logically rigorous, clear direction, fully consistent with context	Label 1 (True Causal)
4	Moderate-Strong Causality	Logically reasonable, correct direction, largely consistent with context	Label 1 (True Causal)
3	Boundary Case	Some association exists, but causal direction is unclear or ambiguous	Discarded
2	Weak Association	Correlation exists but does not constitute a causal relationship	Label 0 (Spurious)
1	No Association	Completely unrelated or logically absurd	Label 0 (Spurious)

This rubric is designed to create a clear separation between true causal relationships and non-causal ones. Notably, samples scoring 3 are treated as boundary cases and are excluded from the dataset entirely, as discussed in the following subsection.

3.3.3 Binarization Strategy and Dataset Construction

To convert the continuous scores into binary labels suitable for training the geometric probe, we adopt a hard thresholding strategy:

- Score ≥ 4 → Label 1 (positive sample, true causal edge)
- Score ≤ 2 → Label 0 (negative sample, spurious or non-causal edge)
- Score = 3 → Discarded (ambiguous boundary case)

We use solely 0 and 1 represent the feature of the sample, instead of setting middle value to represent mediate situations. Since the geometric probe is designed to learn a clear and sharp decision boundary in the embedding space. Soft labels (e.g., using 0.5 or 0.7 as target probabilities) would blur this boundary and reduce the discriminative power of the probe. By using only high-confidence positive and negative samples, we ensure that the probe learns from well-separated geometric patterns.

The table summarizes the dataset statistics at each stage of the pipeline.

Stage	Total Samples(/steps)	Positive (Label 1)	Negative (Label 0)	Pos:Neg Ratio
Raw Steps (Phase I)	5,724	—	—	—
After LLM Filtering	4,293	3,327	966	3.4 : 1
Balanced	1992	966	966	1:1
Training Set (80%)	1592	796	796	1:1
Validation Set (10%)	200	100	100	1:1
Test Set (10%)	200	100	100	1:1

The resulting data set shows a positive-to-negative ratio of approximately 3.4:1 (3,327 vs. 966). Therefore, we drop the the extra positive samples, training a model by using balanced amount of negative samples and positive samples

3.3.4 Human Justification

In all the judgement made by the LLM judge, we randomly selected 10 complete chains (consisting of 64 reasoning steps, 1.5% among all 4293 samples. For my apology we don't have time to label more samples. We supposed to label 300 samples by ourselves to justify the quality of data) along with their corresponding scores. Then, in a double-blind manner, the researchers reviewed the actual data and conducted manual rationality verification. Out of the 64 steps, 59 of them fall within same label (positive & negative), yielding an agreement rate of 92.1% when evaluated by both parties. Although LLM has the risk of transmitting hallucinations to the dataset, in terms of the sampling results, in this study, LLM judge was able to significantly reduce the annotation time while still maintaining a certain level of sample quality.

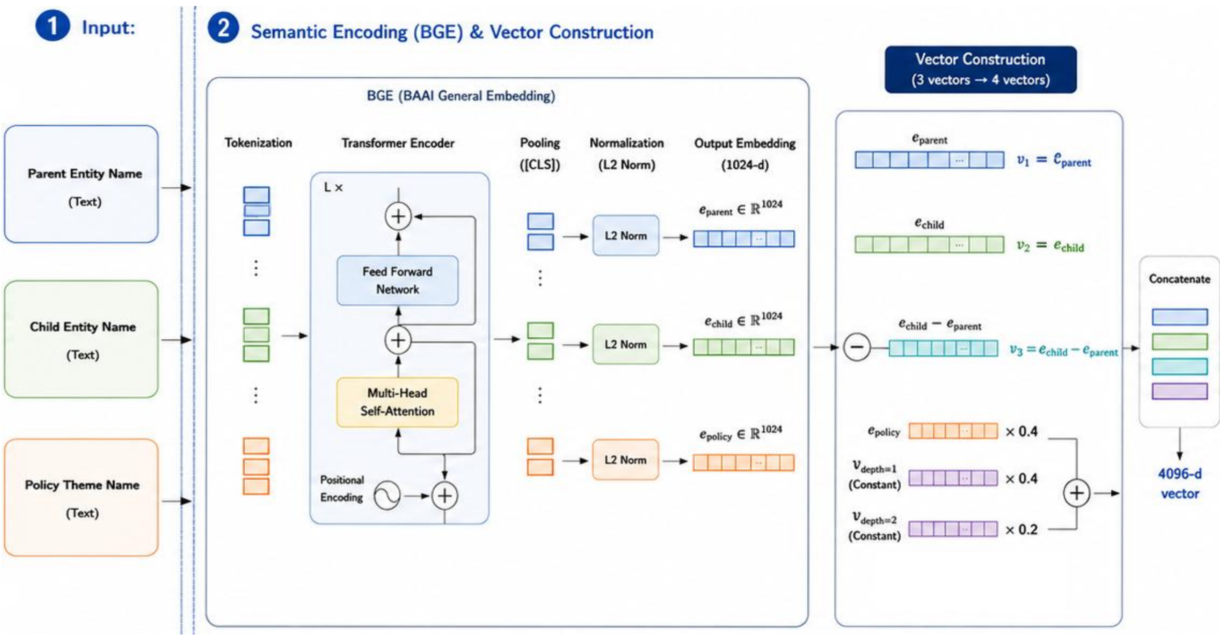
4 Methodology: Geometry-Aware Probe

4.1 Overview: From Text Matching to Geometric Pattern Recognition

In order to achieve the function mentioned, we must declare the essence of casualty. Sense of logic between two things is not only attribute to their similarity of external meanings. In fact, logic between words is complex and implicit, consisting of unknown number of inherent patterns, which computers cannot completely recognize those patterns by merely relying on the rule which was actively set by humans.

In the previous research (A semi-automated approach to policy-relevant evidence synthesis: combining natural language processing, causal mapping, and graph analytics for public policy), researcher design a pattern to extract impacts, agents and patients, forming a A-do-something-to-B pattern. However, two entities which don't have impact words externally in between may also have casualties and logic relationships. The artificially designed standards cannot encompass all the logically implied patterns. Therefore, we should not set a standard ourselves but let computer algorithm learn this standard from a large number of samples.

In this research, we will vectorize words from reasoning chains which are produces by the Policy Chain Agent, and we will use MLP to let computer learn the inherent patterns from arrays of vectors. Then, we will use the model detect the logical pattern in the reasoning chain to decide whether there is logic between two steps of reasoning. The model will detect whether the agent has an Semantic Drift and control whether the agent should keep searching.



4.1.1 Vectorizing with BGE

To represent the textual entities within the causal reasoning chains, we employ the BGE (BAAI General Embedding) model (Xiao et al., 2023) to project each text segment into a continuous semantic space. We select BGE due to its state-of-the-art (SOTA) performance on the Massive Text Embedding Benchmark (MTEB) and its Chinese counterpart (C-MTEB), which demonstrates its superior capability in capturing deep semantic relationships and contextual nuances. Formally, for any given text entity e, its semantic representation is obtained via the BGE encoder as:

$$v_e = \mathrm{BGE}(e) \in \mathbb{R}^{1024}$$

where v_e is a 1024-dimensional dense vector that encapsulates the semantic essence of the input text.

4.2 Four-Dimensional Geometric Feature System

The reasoning chain will be separated into single reasoning steps, and then BGE will vectorize all entities' names into vector that has 1024 dimensions. One single step of reasoning is the sample in MLP training. The data structure consists of four types of vector: vectors of entity n (child entity), vectors of entity n-1(parent entity), the semantic drifting vector, and the semantic displacing vector. The parent entity represent the origin of the reasoning, and the child entity represent the end of the reasoning. Through the comparison of two vectors, we can demonstrate how far the semantics displaces while reasoning, which is the semantic displacing vector. We can also demonstrate how far the semantics drift from the main theme (the semantic drifting vector), by evaluating the difference of the child vector and the central vector, a vector that can be calculated from vector of policy theme and the vectors of entity 1 & 2.

These four 1024-dimensional vectors represent four major features in logical patterns, which we can use to judging whether the reasoning chain has a semantic drift.

4.2.1 Feature 1 & 2: Semantic Coordinates: child & parent vector

To capture the semantic dynamics inherent in causal reasoning chains, we utilize the absolute semantic coordinates of both the parent and child entities as two fundamental features for logic pattern recognition.

The first feature,

$$v_{parent} = \mathrm{Embed}(e_{parent}) \in \mathbb{R}^{1024}$$

represents the semantic coordinate of the reasoning origin, corresponding to the antecedent entity in the 1024-dimensional embedding space. It serves as the contextual anchor that establishes the initial semantic state from which a logical inference proceeds.

The second feature,

$$\mathrm{v}_{\mathrm{child}} = \mathrm{Embed}(\mathrm{e}_{\mathrm{child}}) \in \mathbb{R}^{1024}$$

denotes the semantic coordinate of the reasoning terminus, representing the consequent entity's absolute position within the same latent space. Together, these two features are indispensable for understanding logic patterns because they explicitly encode the semantic boundaries of a single reasoning step. This is critical because the same displacement vector can correspond to different logical relations depending on where it occurs in the semantic space. For instance, a similar semantic shift between "heavy rain" → "flood" and between "studying hard" → "improved grades" may be numerically comparable, yet they represent fundamentally different causal mechanisms—physical causation versus behavioral causation. Retaining the absolute coordinates $\mathrm{v}_{\mathrm{parent}}$ and $\mathrm{v}_{\mathrm{child}}$ allows the model to distinguish such cases by knowing the semantic region in which the shift takes place.

4.2.2 Feature 3: Semantic Displacement Vector

The third feature,

$$v_{displacement} = v_{child} - v_{parent} \in \mathbb{R}^{1024}$$

is the semantic displacement vector that captures how far and in what direction the reasoning moves from one entity to the next in the embedding space. Rather than describing where the entities are located (as v_{parent} and v_{child} do), this feature directly measures the semantic transition that occurs within a single reasoning step.

The logic behind $v_{displacement}$ is straightforward: genuine causal reasoning tends to proceed through small, coherent semantic transitions, whereas spurious or fabricated reasoning often involves abrupt semantic jumps. Consider a valid causal chain such as "smoking" → "lung damage" → "respiratory failure"—each step involves a modest and natural semantic shift. In contrast, a spurious link such as "smoking" → "stock market crash" would produce a displacement vector with a much larger magnitude, reflecting an unnatural semantic leap that lacks plausible causal grounding.

Specifically, we assume that displacement vectors in valid causal chains exhibit two distinguishing properties: (1) relatively small magnitudes, indicating that the semantic change between consecutive entities is gradual rather than abrupt, and (2) directional consistency, meaning the displacement vector aligns with the overall trajectory of the reasoning chain. Conversely, spurious causal links tend to produce displacement vectors with abnormally large magnitudes and directions that deviate from the chain's overall reasoning path, signaling a break in logical coherence.

This design is based on the findings of Ethayarajh (2019) [13], who demonstrated that semantic variation in embedding spaces tends to concentrate along specific directions. Building on this insight, v_{diff} enables the model to evaluate whether a reasoning step represents a natural, directionally coherent semantic transition or an anomalous jump that is inconsistent with plausible causal logic.

4.2.3 Feature 4: Deviation from Multi-Component Centroid

The fourth feature,

$$v_{\text{drift}} = v_{child} - v_{center} \in \mathbb{R}^{1024}$$

measures the degree of semantic drift within the reasoning chain. While the previous three features focus on the local properties of a single reasoning step—where it starts, where it ends, and how it displaces — $\mathrm{v}_{\text{drift}}$ introduces a global perspective by evaluating the current node against the chain's dynamic semantic centroid, $\mathrm{v}_{\text{center}}$, which represent the theme of the entire reasoning chain.

Geometrically, this feature captures how far the current reasoning step deviates from the conceptual "center of gravity" established by the preceding chain. In plain terms, it acts as a drift detector, answering a critical question: is the reasoning drifting off course?

The primary purpose of $\mathrm{v}_{\text{drift}}$ is to identify cumulative semantic drift that local features might miss. A spurious or hallucinated reasoning chain might consist of individual steps that appear locally plausible, yet gradually wander away from the original topic over multiple hops. For example, a fabricated chain might start with "battery overheating," drift to "summer heat," and eventually end up at "ice cream sales." While each adjacent step might have a seemingly reasonable displacement vector ($\mathrm{v}_{\text{displacement}}$), the overall chain has clearly lost its logical focus. By measuring the distance between the current entity and the dynamic centroid, $\mathrm{v}_{\text{drift}}$ explicitly quantifies this deviation, ensuring that the reasoning remains tightly anchored to its core causal theme.

The specific computation of the dynamic centroid $\mathrm{v}_{\text{center}}$ is a core innovation of our approach and will be detailed in Section 4.3.

4.3 Multi-component Central Semantic Vector

In this section, we introduce the Multi-component Central Semantic Vector, denoted as v_{center}. Its primary purpose is to serve as a reliable reference point for measuring semantic drift during the generation of causal reasoning chains. Instead of relying on a single fixed concept (such as the initial policy topic), v_{center} is constructed by combining the embedding vectors of the policy topic and the first two reasoning steps (v_{depth1} and v_{depth2}). By integrating these early steps, v_{center} provides a more grounded baseline that captures the specific analytical direction of the current chain. This allows the model to accurately evaluate whether subsequent reasoning steps represent valid, detailed extensions or true off-topic deviations, thereby improving the overall coherence and relevance of the generated chains.

4.3.1 Problem Motivation: Limitations of Single-component Anchors

To understand why a multi-component vector is necessary, we must first examine the limitations of using a single-component anchor, such as the initial policy vector (v_{policy}) alone.

The core issue is an abstraction-level mismatch. The policy topic is highly abstract and macro-level. However, the nature of causal reasoning is to move from macro concepts to micro, concrete entities. As the chain goes deeper, the reasoning naturally becomes more specific. If we only use v_{policy} as the reference point, the semantic distance between the policy and the deeper nodes will inevitably grow, even if the reasoning is perfectly logical.

Consider a concrete example. Suppose the initial policy is "Housing purchase restrictions are relaxed" (v_{policy}).

- A valid first step (v_{depth1}) might be: "Housing sales volume increases."

- A valid second step (v_{depth2}) might be: "Demand for building materials rises."

- Now, suppose the model generates a third step (v_{child}): "Sales of M8 steel bolts for construction grow significantly."

Logically, this third step is a valid, detailed extension of the building materials theme. However, semantically, the concept of "M8 steel bolts" is extremely far from the abstract concept of "housing purchase restrictions" in the embedding space. If we only measure the drift against v_{policy}, the distance will be very large. The model might mistakenly treat this valid refinement as semantic drift and penalize it (a false positive). This happens because the single anchor v_{policy} lacks the context of how the chain has progressed; it only knows the starting point, not the established direction.

Adding component from the chain can solve this problem.

By incorporating v_{depth1} and v_{depth2} into the calculation ($v_{center} = \alpha \cdot v_{policy} + \beta \cdot v_{depth1} + \gamma \cdot v_{depth2}$), the central vector gains a clear directionality. The first two steps determine the "main channel" or the specific analytical perspective of the chain (in this case, shifting from housing policy to physical construction materials).

When

v_{center} includes this directional context, it shifts from a highly abstract origin to a more concrete baseline that aligns with the chain's current focus. Consequently, when we evaluate the "M8 steel bolts" node against v_{center}, the measured drift is much smaller and closer to its true semantic deviation. Conversely, if a node genuinely drifts off-topic (e.g., suddenly discussing "cryptocurrency prices"), it will deviate not just from the policy, but from the entire established context of housing and construction materials, resulting in a sharply high drift score against v_{center}.

Overall, the first two steps define the overall trajectory of the reasoning chain. By including them in the central vector, we reduce the false rejection of valid, detailed reasoning and ensure that the measured semantic drift accurately reflects true deviations rather than natural semantic refinement.

4.3.2 Formulation

This is the formulation design to calculate v_{center}:

$$v_{center} = \alpha \times v_{policy} + \beta \times v_{depth=1} + \delta \times v_{depth=2}$$

which α, β and δ are the weights of the three vectors. Specifically:

$$\alpha + \beta + \delta = 1$$

In this research, the three weights in sequence is 0.4, 0.4 and 0.2, which the formulation will be like:

$$v_{center} = 0.4 \times v_{policy} + 0.4 \times v_{depth=1} + 0.2 \times v_{depth=2}$$

4.3.3 Weight Design Rationale

Each vector & weight design has its purpose:

vector	weight	meanings
v_{policy}	0.4	Macro-semantic constraints
$v_{depth=1}$	0.4	Main trunk direction anchoring
$v_{depth=2}$	0.2	Local inertial trend

v_{policy} represent the semantic of the general theme of the reasoning. This component keeps v_{center} from being divorced from the general thesis. However, the weight of v_{policy} cannot become very high, or it will have less difference with Single-component Anchors, a kind of v_{center} that has general semantics, which will be more likely to judge low-abstract-level entity as semantic drift. The magnitude of the weight cannot approach 1,if so, the model will mark all the step of reasoning as “semantic drift” because the name of the policy itself is a macroscopic concept. As the reasoning progresses, the entities affected might shift from a macroscopic concept to a microscopic one. If the semantic vector weight of the policy name is too large, then all such transitions will be labeled as semantic drift or illusion, causing wrong judgement.

The entity at depth=1 is the direct receptor of reasoning and also the starting point of reasoning. The content of this entity mainly determines the direction and perspective of the reasoning chain. The major effect of $v_{depth=1}$ is to provide contextual semantic to the feature for every single . Compared to a globally unified single-component anchor point, this is more accurate. Additionally, by giving it the same weight as the v_{policy}, the chain neither deviates from the macro theme nor lacks a micro perspective that is more in line with the specific content.

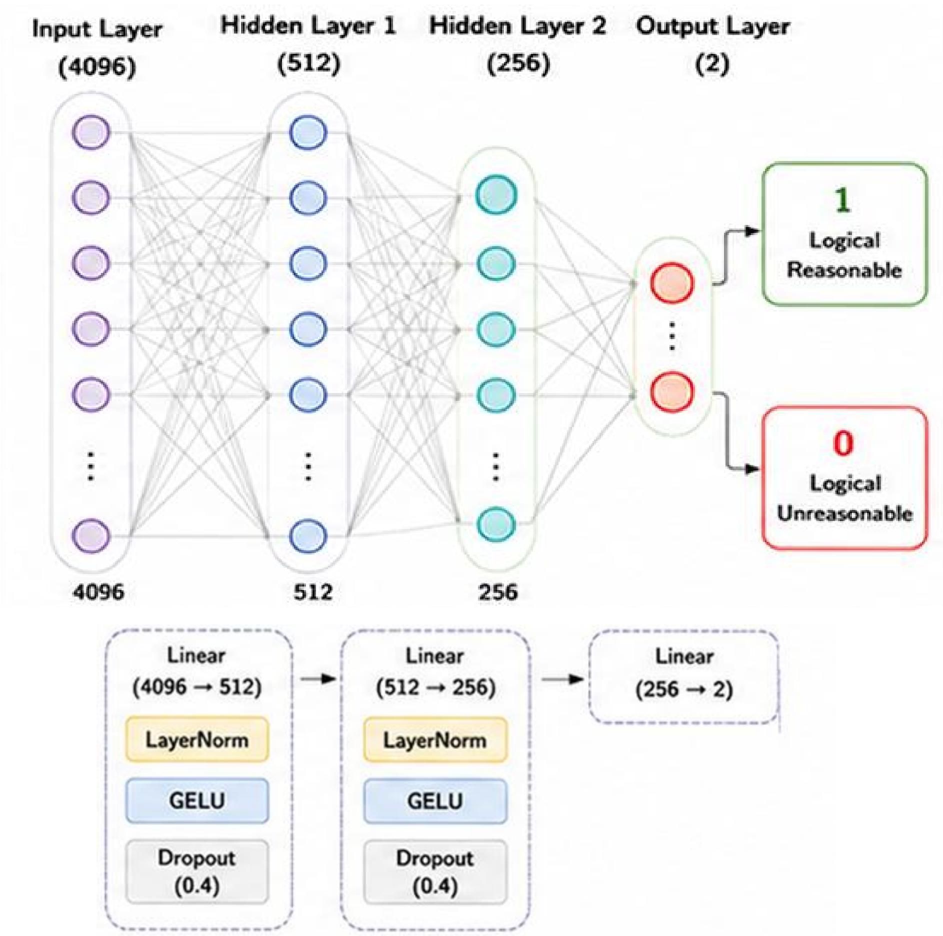
The influence on entity at depth = 2 is derived from the influence on entity at depth=1, and entity at depth=2 often determine what is the chain exactly related to. By adding $v_{depth=2}$ into the formula, v_{center} can catch the chain's trend of direction. This component also provides context to the feature, but the information provided is more specific and detailed. However we can't ensure that our agent can eliminate all the noise, the interference information, in the non-official web side. Therefore, entity from depth =2 to depth = n may all be polluted by noise, which the noise will change the direction of the actual semantic vector and affect the result . Thus we do not give this component a high weight, to reduce the effect of noise.

We must mentioned the limitation of this part of work . The three weight 0.4 0.4 and 0.2 are define by human intuition. We have not evaluate the best weight combination by doing more experiments.

4.4 Model Architecture: Lightweight Multi-Layer Perceptron (MLP)

To evaluate the reasoning hallucination of generated nodes, we design a lightweight Multi-Layer Perceptron (MLP) as the scoring probe. The goal of this probe is not to learn complex language representations from scratch, but to map our pre-computed geometric features into a binary classification decision (valid extension vs. semantic drift). The binary classification will output a value from 0 to 1, which shows the confidence level of "whether this step of reasoning is a reasoning hallucination".

4.4.1 Network Structure



The architecture of our MLP is intentionally simple and shallow. It takes the concatenated 4,096-dimensional feature vector as input and processes it through two hidden layers before outputting a 2-dimensional logit for binary classification. The detailed structure is as follows:

- Input Layer: $x \in \mathbb{R}^{4096}$ (the concatenation of the four geometric features: v_{diff}, v_{center_diff}, v_{child}, and v_{center}).

- Hidden Layer 1: Linear transformation ($4096 \rightarrow 512$), followed by Layer Normalization, GELU activation, and Dropout ($p = 0.3$).

- Hidden Layer 2: Linear transformation ($512 \rightarrow 256$), followed by Layer Normalization, GELU activation, and Dropout ($p = 0.3$).

- Output Layer: Linear transformation ($256 \rightarrow 2$) to produce the final classification logits.

The total number of trainable parameters in this network is approximately 2.2 million ($4096 \times 512 + 512 \times 256 + 256 \times 2 \approx 2.2M$).

4.4.2 Reason for Choosing MLP

We chose a simple MLP over more complex architectures, such as Transformers or LSTMs, based on the principle of Occam's Razor: "Entities should not be multiplied beyond necessity." When a simple model can achieve over 90% accuracy, introducing a complex architecture, such as Transformer or LSTM, might cause decline on performance .

We choose MLP for the model structure, in a reason that features are already fully encoded, and the core information about causality and logic is already captured by our four-dimensional geometric features. Because these input features are highly separable and informative, a deep network with self-attention mechanisms is simply redundant.

Additionally, the use of simple structure is to prevent overfitting on small data. Our training dataset contains only 1992 samples. Training a complex model with millions of extra parameters on such a small dataset almost guarantees severe overfitting. In our preliminary experiments, the MLP reached its optimal performance in the very first epoch, while deeper models struggled to generalize (in fact, the result in chapter 6 shows that Transformer or LSTM might be a better choice).

4.5 Training Configuration

To train the MLP probe effectively while strictly preventing overfitting on our limited dataset, we carefully selected the training hyperparameters. The detailed configuration and the rationale for each choice are summarized in the table.

Hyperparameter	**Setting**	**Rationale**
Optimizer	AdamW	Standard choice for stable convergence, incorporating weight decay for better generalization.
Learning Rate	2e-4	Paired with a cosine annealing scheduler to smoothly reduce the learning rate and avoid local minima.
Batch Size	64	Balances gradient stability and training speed for our dataset size.
Dropout Rate	0.3	Applied after each hidden layer to randomly drop neurons and prevent the network from relying too heavily on specific features.
Weight Decay	1e-4	Acts as L2 regularization to penalize large weights and further reduce the risk of overfitting.
Loss Function	Cross-Entropy Loss	The standard and most effective loss function for binary classification tasks.
Early Stopping	patience = 5	Monitors the validation loss. If the validation loss does not improve for

		5 consecutive epochs, training stops to prevent the model from memorizing the training data.

5 experiment

5.1 Experimental Setup

5.1.1 Evaluation Metrics

We use five metrics to evaluate model performance. Especially,Macro-F1 treats both negative and positive classes equally because the two benchmarks show different kind of ability ---- detecting hallucination and identifying valid logic.

5.1.2 Baseline Methods

We design four baseline methods to test different aspects of our approach.

We establish the absolute lower performance bound using a feature-free baselines which the Uniform Random predicts a 50/50 positive-negative split.

Cosine Similarity is one of the previous method for detecting reasoning hallucination. So we use this method as a baseline that tests whether simple semantic similarity is enough to detect causal relations. We compute the cosine similarity between the parent vector and the child vector: $\cos(v_{parent}, v_{child})$. We then search for an optimal threshold on the validation set that maximizes Macro-F1. If the similarity is above the threshold, the pair is predicted as causal; otherwise, it is predicted as non-causal. This baseline uses no model parameters and no policy context.

Static Anchor (Ablation) is the original solution for providing context in our research. Our model computes a dynamic anchor as a weighted combination:

$$v_{anchor} = 0.4 \times v_{policy} + 0.4 \times v_{d1} + 0.2 \times v_{d2}$$

But the Static Anchor variant removes this multi-component weight mechanism ($0.4 \times v_{d1} + 0.2 \times v_{d2}$) and uses only v_{policy} as a fixed anchor. We then compute the same four geometric inputs (parent, child, diff, deviation from anchor) and feed them into the same MLP architecture. The model is trained from scratch with identical hyperparameters. This experiment isolates the contribution of the dynamic weighting mechanism.

We use DeepSeek V4 Pro as a zero-shot causal classifier, providing it with the policy context, parent event text, and child event text to output a binary causal judgment. This baseline evaluates whether a powerful LLM can detect causality without task-specific training, while also recording inference time and API cost for efficiency comparison.

5.2 Main Results

5.2.1 Performance of the hallucination detecting MLP

We evaluated the model on the test set (N = 430).

Metric	Value
Accuracy	91.63%
Macro-F1	89.13%

This table show the model 's performance on positive samples and negative samples.

Class	Precision	Recall	F1-score
True Logic (Label = 1)	0.94	0.88	0.9091
Reasoning Hallucination (Label = 0)	0.88	0.95	0.9154

The result shows that the model have had basic capacity for detecting both valid logic and reasoning hallucination.

5.2.2 Baseline Comparison

This table presents the full comparison across all methods and then comparison of the GeometryAwareMLP against baseline methods on the test set (N = 430).

Model	Accuracy	Macro-F1	Neg-F1	Pos-F1	Parameters	Inference / Sample
Random (Uniform)	0.5023	0.4616	0.3140	—	0	0 ms
Cosine Similarity	0.6163	0.4751	0.2029	0.7473	0	**< 1 ms**
Static Anchor	0.8419	0.8083	0.7280	0.8885	2.2M	< 5 ms
LLM-as-a-Judge (DeepSeek V4 pro)	0.7000	0.6940	0.7368	0.6512	—	~2 s / $0.00002
Multi-component Anchor	**0.9163**	**0.8913**	**0.8393**	**0.9434**	2.2M	< 5 ms

The Random (Uniform) baseline achieves a Macro-F1 of 0.4616. Our model achieves 0.8702, a relative improvement of 89%. This large gap confirms that the model learns real causal patterns from the geometric features, rather than guessing or exploiting class imbalance.

Cosine Similarity achieves a Neg-F1 of only 0.2029. This means it almost completely fails to detect pseudo-causal pairs. The reason is that many non-causal event pairs are still semantically similar. For example, two events may share keywords or discuss the same policy topic without having a true cause-effect relation. Cosine Similarity captures semantic overlap but cannot detect the directional and structural properties of causal transmission. Our model, by contrast, uses four geometric perspectives (parent position, child position, displacement vector, and deviation from the dynamic anchor) to capture these deeper structural signals. The Pos-F1 of Cosine Similarity is 0.7473, which is much lower than our 0.9300. This shows that even for true causal pairs, simple similarity is not a reliable indicator.

This is the most important comparison. When we replace the dynamic anchor with a static policy-only anchor, Macro-F1 drops from 0.8702 to 0.8083, a decrease of 0.062 (−7.1%). The drop is even larger for Neg-F1: from 0.8100 to 0.7280, a decrease of 0.082 (−9.9%). Accuracy also drops from 0.9023 to 0.8419.

This result shows that using v_{policy} alone as the anchor is not enough. The policy context vector provides useful background information, but it does not capture the directional properties of the specific causal pair. The dynamic anchor combines policy context ($0.4 \times v_{policy}$) with pair-specific directional signals ($0.4 \times v_{d1} + 0.2 \times v_{d2}$). This combination gives the model a more precise reference point for measuring how much a causal pair deviates from the expected transmission path. The ablation result confirms that the dynamic weighting mechanism is a necessary component of our approach.

The zero-shot LLM (DeepSeek V4 pro) achieves a Macro-F1 of 0.6940, which is substantially lower than our model's 0.8702. The LLM performs especially poorly on positive samples (Pos-F1 = 0.6512 vs. our 0.9300). One possible reason is that the LLM has not been trained on this specific task and tends to be conservative when judging complex policy texts. It may fail to recognize subtle causal links that require domain-specific knowledge.

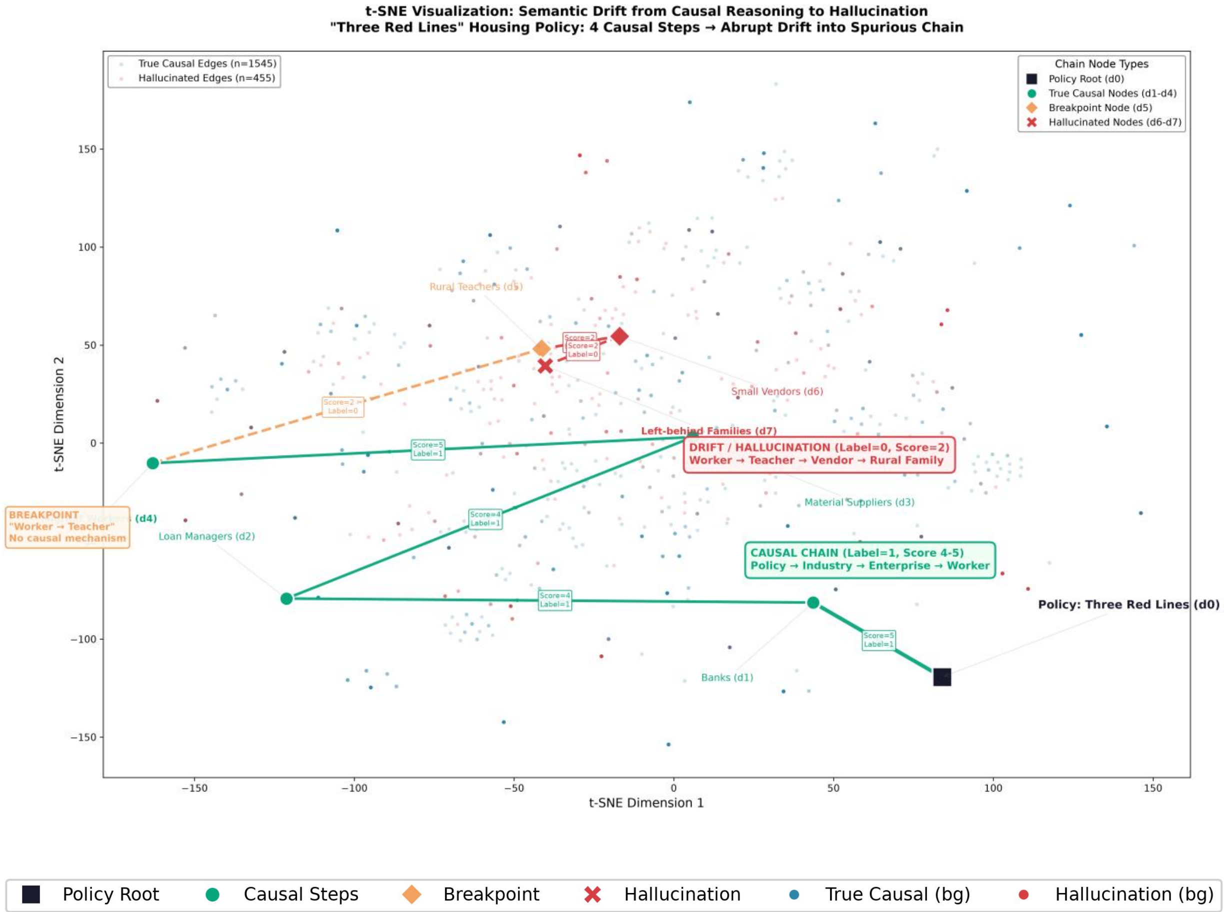
From an efficiency perspective, the LLM requires approximately 2 seconds per sample and incurs an API cost (approximately 0.00002 RMB persample ,or about 0.009 for the full test set of 430 samples). Our model completes inference in under 5 milliseconds per sample, which is approximately 400 times faster, with zero API cost. This makes our model practical for large-scale policy graph construction where thousands or millions of edges need to be evaluated.

The last row of the baseline table compare our two imbalance handling strategies. Our model “Multi-component Anchor” outperforms every baseline across all four metrics. The model preserves the natural geometric distribution of the feature space, allowing the MLP to learn a cleaner causal boundary.

Additionally, we draw t-SNE graph using output data from the output of the hidden layer that has 256 dimensions, which the output vectors has already been processed by most of the neurons so we can discover common patterns from solely observing the graph. This graph takes 256-dimensional vectors that have already been analyzed by the model to reveal the patterns, and then reduces the dimensions to visualize the underlying rules.

Here is an example about the chain:

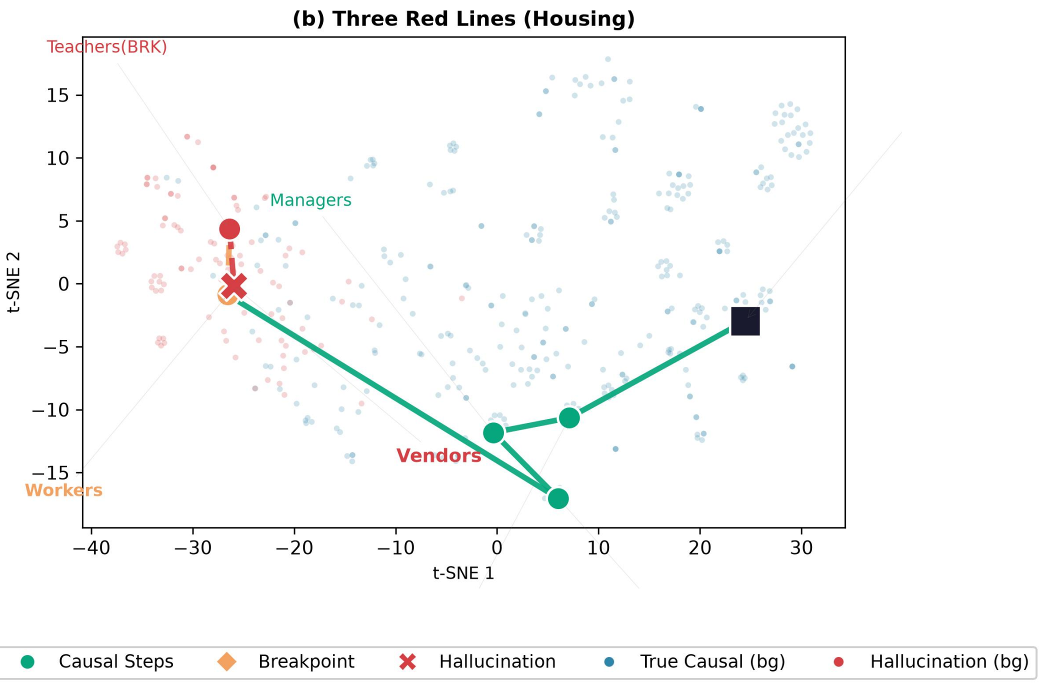
3 Red Lines → Banks → Loan Mgrs → Suppliers → Workers -(hallucination)->Teachers → Vendors → Benefits



Every node represent a step of reasoning in a chain. The cross node represent the step which is marked as reasoning hallucination. The black node represent the policy theme, which is the origin.

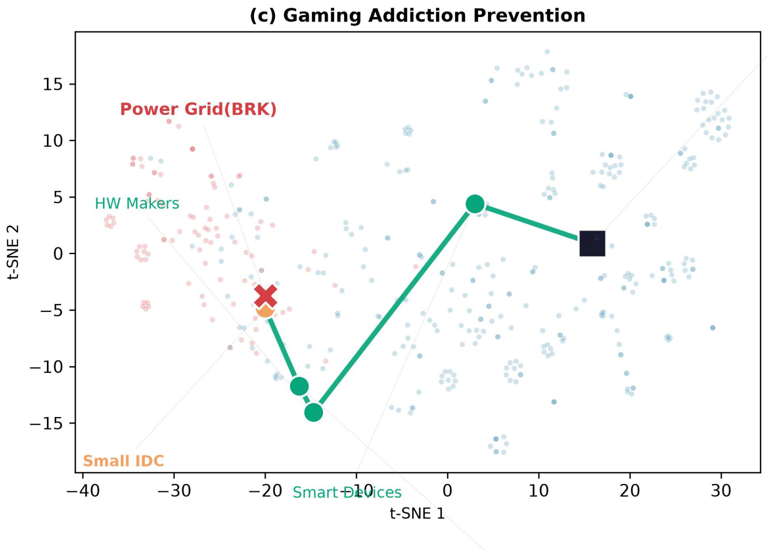
The first graph shows how semantic displace in the reasoning chain when the semantic vector haven't been processed by the model yet. Before the reasoning information has been processed by the model, we cannot observe any common features in the tSNE graph that could reflect the occurrence of reasoning hallucination.

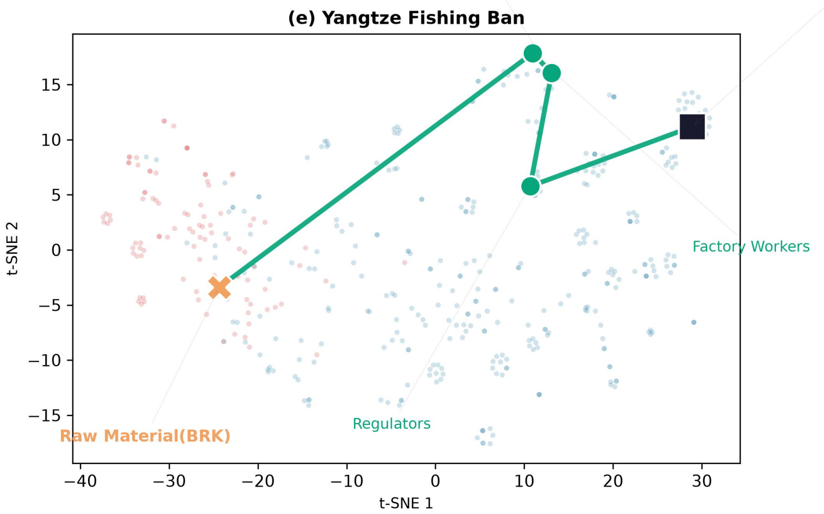
The next graph shows the displacement of the semantics after the reasoning step is being processed by the model.



We can observe that in all reasoning steps (the node) processed by the model, semantics of the reasoning hallucination (the cross node) are generally farther from the policy theme node. However, after we input the reasoning data into the model for loading, we discovered common features that enabled us to visualize the occurrence of reasoning hallucinations – the distances between nodes in the tSNE graph.

We can also find this pattern in other reasoning chains that have been processed by the model.





As the node goes father from the policy node, the model will be more likely to judge the step of reasoning as “reasoning hallucination”. This pattern is not simple cosine similarity, this is an implicit pattern that can only be visualized after being refined by our model. Our model can identify the occurrence of reasoning hallucination through the geometric features in the embedding space. So, at least in familiar policy topics, our model have discovered the pattern of reasoning hallucination.

5.3 Summary

Our experiments yield three core conclusions. First, our model significantly outperforms all baselines in reasoning hallucination classification (Macro-F1: 89.13%), proving its robust detection capability. Second, the architectural design is fundamentally effective. Multi-perspective geometric features vastly outperform simple similarity metrics—especially in identifying pseudo-causal pairs—and the multi-component weight mechanism is critical to maintaining high performance. The rapid single-epoch convergence further confirms that these geometric features provide a highly informative input space. Finally, compared to zero-shot LLMs, our approach offers overwhelming practical advantages. It not only achieves superior accuracy (Macro-F1 0.87 vs. 0.69) but also drastically reduces inference time from 2 seconds to 5 milliseconds per sample while completely eliminating API costs, making it highly efficient and scalable for real-world deployment.

Despite these strong results, it remains uncertain whether the model can maintain this level of performance in actual use.

6 Actual Test Result

We embed the model into the reasoning agent, and test whether the model can actually detect LLM's hallucination in general scenario. We randomly selected a policy theme which hasn't been used for mining neither training dataset nor testing dataset. Unfortunately, the performance of the model in this test is surprisingly subpar.

We deployed the model on the unseen "Export Tax Rebate" policy. The model processed 36 reasoning chains, yielding a much more reasonable distribution: 24 chains

(66.7%) passed completely, while 12 chains (33.3%) were truncated.

The model marked two step of reasoning as “reasoning hallucination”, but in fact, the reasoning steps are valid. Here are the two step of reasoning:

High energy-consuming industries (aluminum electrolysis, ferroalloys, yellow phosphorus, calcium carbide, etc.) {depth=2} **--X-->** [P = 0.34] **Farmers around the yellow phosphorus production area in Qujing City, Yunnan Province** {depth=3}

High energy-consuming industries (aluminum electrolysis, ferroalloys, yellow phosphorus, calcium carbide, etc.) {depth=2} **--X-->** [P = 0.26] **Residents around iron alloy smelting enterprises in Baise City, Guangxi Province** {depth=3}

For the first example, we find facts which are contradicted to the result of the judgement. In upper-Yangtze yellow-phosphorus plants, illegal “sky-lantern” gas flaring and phosphorus-laden wastewater discharges have destroyed fish ponds and crops, directly harming adjacent farming communities (CCTV Focus Interview, 2019; CCTV News Broadcast, 2019).

The second example is similar to the first two. We also found facts that contradicted the judgment made by this model (Baise Municipal Ecology and Environment Bureau, 2026). The reference is also about environmental pollution.

Besides, our model perform unstable on identifying multi-hop logic in one single steps. For the chain Rare earth industry → Residents around mining areas, the model decisively rejected it with a extremely low score:**P=0.024**. On the second "invalid chain", High energy-consuming industries → Farmers around the yellow phosphorus production area in Qujing City, Yunnan Province, the model gives out an much lower score, **P=0.180**. In order to prove the ability of the model, both scores must be greater than 0.6.

Our model also made mistakes on identifying simple logic. Such as the two chains:

Photovoltaic Industry {depth=2} **--X-->** [P = 0.046] **Domestic Photovoltaic Power Station Investors**

Photovoltaic Industry {depth=2} **--X-->**[P= 0.283] **Overseas Photovoltaic Project Developers**

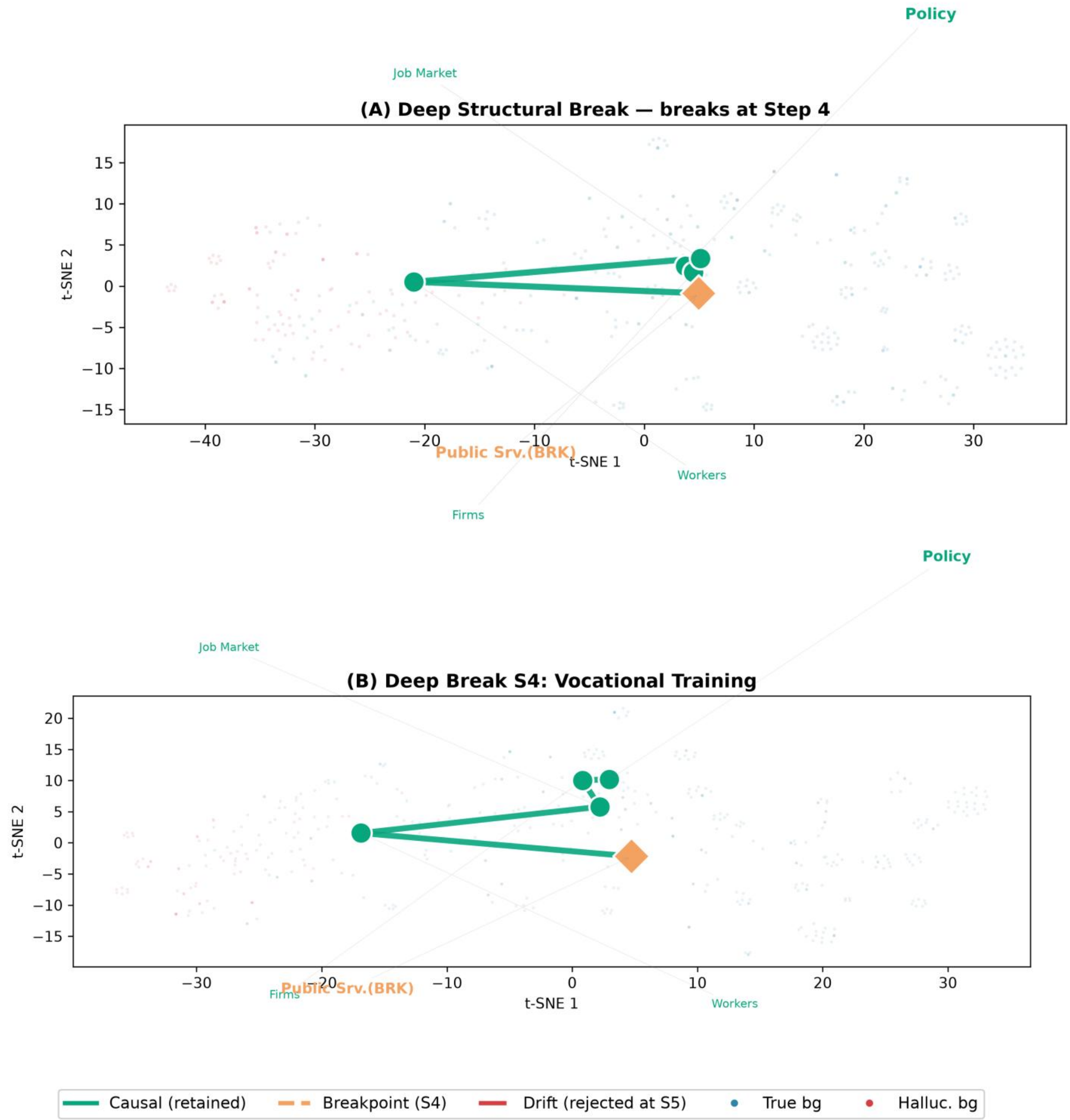
This judgement is an obvious mistakes, which we can even observe the external logic relation between the two entities. The two chains don't have multi-hop logics, but the model still marks them as "invalid".

During the test, the model consistently rejected all 5 reasoning chains starting with **Export tax rebate** → **Battery industry** (all Step 1 P < 0.26). Human verification with existing evidence confirmed that the judgement of the model on these five chains are incorrect. This might be because the policy topics in the training samples did not cover the relevant policy topics of the upstream electronic industry in the battery sector, which led to the model lacking judgment in this field.

We also visualize the semantic distribution in t-SNE (t-Distributed Stochastic Neighbor

Embedding) graph, using data from the output of the hidden layer that has 256 dimensions. In this layer, the feature has already been processed by most of the neurons, therefore whether the distribution shows patterns reveals if the model can detect logic and reasoning hallucination.

Here are some graph showing how semantics displaces in a four-steps reasoning chain.



The graph reveals that the chain was cut off at step 4 (the orange dot) which might be the reasoning hallucination. Each dot in the chain shows the semantic position of reasoning steps in embedding space.

However, in this graph, and every other graph in test result, we didn’t find any common patterns showing that “in which relative position the dot will be marked as hallucination”. This fact shows that our model solely learned the logical patterns in policies that the model familiar with, but cannot identify logical patterns in unfamiliar topics.

6.3 Limitations and Assumptions

Due to the time constraint, we were unable to identify the cause of this problem through more extensive experiments. We will expound the limitations and our assumptions to the problem, by observing the existing samples, which aim to inspire subsequent research.

The lack of the data scale also limits the capacity of the model. From all experiments’ result, the model performs on policy themes which have already included in the training data set, but it frequently makes mistakes on themes that it haven’t seen yet.

Low qualities of data might be another huge limitation. This research relies on the LLM-as-judge mechanism, where the quality of the training data set can be hugely affected by the assessment standard prompt. In the labeling of this non-numerical dataset, human participation in evaluation is an important measure to enhance data quality. However, in this study, there was a shortage of manpower, so only sampling inspection could be carried out. Almost all the work in sample labeling was completed by LLM. These factors all decrease the data quality.

Moreover, the hallucination and bias of the LLM judge will transmit to the MLP model through scoring. Ironically, LLM's hallucination itself may affect the quality of data for training a model which is used to detect hallucination. Specifically, if the LLM judge fails to recognize deep sociological causality and labels it as spurious, our probe will learn to replicate this "LLM-perceived spurious pattern" rather than identifying genuine logical flaws, thereby inheriting the judge's blind spots when facing complex real-world scenarios.

In the feature engineering of this research, the formula $v_{center} = 0.4 \times v_{policy} + 0.4 \times v_{depth=1} + 0.2 \times v_{depth=2}$ consists of three important weights and three crucial components for considering the semantics. However, the hyperparameters are intuitive and have few references. The multi-component mechanism is to create context for each reasoning chain, forcing the model to learn logical patterns under context. The research has not tried to evaluate the optimal weight combination through more experiments yet. More importantly, the current weight assignment explicitly down-weights the semantic information of depth=2 (only 0.2) to reduce the effect of noise. While this design protects the centroid from being polluted by noisy deep nodes, it may also directly causes the model to struggles at step 2 in real-world tests: the model lacks sufficient semantic reference from the depth=2 context to accurately judge the validity of the transition, making it more prone to aggressive truncation at this specific depth.

Another limitation might be that MLP is a simple model which the parameters are too little to learn implicit logical patterns from samples. Thus the model is difficult to satisfy the requirement of detecting reasoning hallucinations. At the beginning of the research, we assume that we have integrally extract features so we used simple-structure model like MLP to prevent overfitting. However, the result of the experiments contradicts to our assumption. The capability of MLP on understanding implicit logic are still limited to achieve detecting hallucinations.

When there are changes in the level of abstraction (from macro-level industry to micro-level specific regional groups), the geometric pattern representing the logical patterns in the semantic embedding space will be extremely complex and become hard to recognize. This is because a cross-level transition naturally produces a large semantic displacement in the embedding space, regardless of whether the causal link is valid or spurious. The model must find out the "logical connection" between two entities that have different perspectives, semantics, and abstract levels, solely from the observation of the 1024-dimensional vectors. Even though we tried our best to extract accurate features from the samples, the implicit logical patterns hidden behind the reasoning are still extremely complex for the MLP model to interpret. The geometric probe can capture statistical regularities of reasoning hallucination, but it cannot substitute for the domain-specific world knowledge (e.g., knowing that Baise is indeed a major ferroalloy production base) required to evaluate deep sociological causality.

7 Conclusion

In this research, we employ text embedding and extract features from textual vectors. Then, by using an MLP model, we attempt to learn implicit semantic and logical patterns to detect LLM’s reasoning hallucinations.

Conclusively, the model has impressive performance on detecting hallucination in policy theme that they have seen (the benchmark test).

For the question “do geometry reveal logic?”, the answer might not be 100% “yes”. Within the known policy topic, the geometric features of the embedding space can serve as an effective proxy indicator for reasoning illusions; however, when it comes to cross-domain and cross-abstraction-level scenarios, pure geometric features are insufficient to replace world knowledge.. The training data in chapter 5 shows that logic-based detection of hallucination is generally practical, and this field can serve as a future sustainable research direction. However, there are still limitations found in actual test, restricting the model's usage scenario and capabilities. Although semantic vectors do reveals logical patterns, this research still cannot make a final conclusion to ensure that the logic-based method is generally better than the fact-based method for detecting hallucinations. In the next chapter, we will discuss the future approaches and give instructor to the future research.

8 Future Approaches

A primary direction is improving data quality at the source, which is the agent. Currently, the training data relies on LLM-generated reasoning chains and LLM-as-judge labeling. We didn't consider that LLM judge itself will also produce hallucination.

Future research could break this loop by enhancing the agent's denoising and cross-validation capabilities, or by extracting causal reasoning chains directly from existing academic publications and government policy reports. These source of information may be more reliable than our agent. Besides, human should participate deeply on labeling the data. In this study, almost all labeling was done by LLM because the team was working on a hackathon in the same time. Human-validated data would provide the geometric probe with more reliable supervision signals.

The model itself also need optimization. The multi-component anchor ($v\_center = 0.4 \times v\_policy + 0.4 \times v\_depth{=}1 + 0.2 \times v\_depth{=}2$) is intuitive and haven't been tested by any other research in the world. We design central vector to provide context for the judgement.

We have not justify the 0.4/0.4/0.2 weight combination and whether this solution is better.

In fact, light weight Transformer classifier or Graph Neuron Networks (GNN) may providing a more scientific context and have higher performance in processing sequential data than our intuitive model. More specifically, self attention may provide more contextual information than solely using central anchor.

Not only central anchor got its issue, MLP model itself also had its limitation. The MLP has only 2.2M parameters, which is too small to capture complex semantic patterns, in another word, this model is too simple to achieve the goal. In previous experiments, no matter how we tried, the battery anomaly in Section 6.3 shows that the model cannot recognize patterns absent from its training data. This is another reason why we speculate that Transformer will do better. Transformer structure requires lager scale of dataset, and has larger amount of parameters, which it can learn more complex patterns than MLP.

In our perspectives, future work on this issue can concentrate on two major solutions. One is designing a much more complex model which can generalize this implicit logic patterns. Or, another solution might be constructing a huge data base which contain nearly all categories of policy. Our corpus covered only 10 policy themes, leaving blind spots for emerging sectors like batteries and semiconductors. Our model still cannot generalize to the actual scenario. Thus the improving the scale of coverage of data is crucial for the generalization.

In the future, we will first verify the rationality of the multi-component central anchor, and find the best weight combination. We will also improve the quality of the dataset. Then, we will make a comparison between Transformer and our model.

All our codes, dataset, model are open source, so if you have willingness to push forward the research, we can share all our materials to you. We will simultaneously open-source the paper and the project on GitHub, and also attach the technical documentation to facilitate further development.

REFERENCE:

Ackermann, F., & Alexander, J. (2016). Causal mapping for problem structuring. In M. Reynolds & S. Holwell (Eds.), Systems approaches to making change: A practical guide (pp. 201–252). Springer.

Azaria, A., & Mitchell, T. (2023). The internal state of an LLM knows when it's lying. Proceedings of EMNLP 2023.

Baise Municipal Ecology and Environment Bureau. (2026). Bai huan fa [2026] No. 5 / Bai huan ze gai [2026] No. 22: Administrative penalty for environmental violations by Baise Yida Metal Materials Co., Ltd. [Government document]. Baise Municipal Government. https://www.baise.gov.cn/...

Belinkov, Y., & Glass, J. (2019). Analysis methods in neural language processing: A survey. Computational Linguistics, 45(1), 63–105.

Beltagy, I., Lo, K., & Cohan, A. (2019). SciBERT: A pretrained language model for scientific text. Proceedings of EMNLP-IJCNLP 2019, 3615–3620.

Burns, C., Ye, H., Weiss, D., & Belinkov, Y. (2023). Discovering latent knowledge in language models without supervision. International Conference on Learning Representations (ICLR).

CCTV Focus Interview. (2019, July 3). Zhenggai zhenggai, guji nan gai [Rectification, rectification, chronic problems hard to change]

[Video]. CCTV.com. https://tv.cctv.com/2019/07/03/VIDE9qktJzyG8sgpR0JCrRhQ190703.shtml

CCTV News Broadcast. (2019, July 4). Guizhou Weng'an "zheng er bu gai" huanglin qiye wuran yanzhong, weiqi zhipai "dian tian deng" wuran zhoubian huanjing [Guizhou Weng'an: "Rectified but not reformed," yellow phosphorus enterprises cause severe pollution, tail gas directly discharged as "sky lantern" pollutes surrounding environment] [Video]. CCTV.com. https://tv.cctv.com/2019/07/04/VIDEFgKh5vfnfGdPJZLwpbs7190704.shtml

Chen, S., Mai, X., & Xu, J. (2008). The impact of export tax rebate policy adjustment on China's energy-intensive industries: A CGE analysis. China Economic Review, 19(4), 567–580.

China News Network. (2012, April 26). "Xitu wangguo" Gannan de xitu zhi shang ["Rare earth kingdom" Gannan's rare earth tragedy]. https://www.chinanews.com.cn/ny/2012/04-26/3846887.shtml

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. Proceedings of NAACL-HLT 2019, 4171–4186.

Ding, Q., Wu, Y., Shen, Y., et al. (2023). Large language models can evolve from their own mistakes. arXiv preprint.

El-Taliawi, O. G., Howlett, M., & Tan, S. (2021). Policy analytics: A framework for assessing policy capacity. Policy Studies, 42(5–6), 495–514.

Ethayarajh, K. (2019). How contextual are contextualized word representations? Comparing the geometry of BERT, ELMo, and GPT-2 embeddings. Proceedings of EMNLP-IJCNLP 2019, 580–589.

Gao, Y., Xiong, Y., Gao, X., et al. (2023). Retrieval-augmented generation for large language models: A survey. arXiv preprint arXiv:2312.10997.

Goyal, N., & Howlett, M. (2019). Measuring the mix of policy instruments: A systematic framework. Policy Sciences, 52(3), 415–436.

Haddaway, N. R., Woodcock, P., Macura, B., Collins, A., D'Errico, L. O., Hanzee, M. J., & Taylor, J. J. (2017). Evidence for the impacts of agroforestry on agricultural productivity, ecosystem services, and human well-being in high-income countries: A systematic map protocol. Environmental Evidence, 6(1), 1–13.

Hansen, H. F., Nilsson, M., & Rieper, O. (2022). The role of systematic reviews in evidence-informed policymaking. Evidence & Policy, 18(2), 285–302.

Huang, L., Yu, J., Ma, W., et al. (2023). A survey on hallucination in large language models: Principles, taxonomy, and methods. arXiv preprint arXiv:2311.05232.

Ji, Z., Lee, N., Frieske, R., et al. (2023). Survey of hallucination in natural language generation. ACM Computing Surveys, 55(12), 1–38.

Koleski, C. (2012). The Chinese rare earth monopoly: Implications for U.S. national security and economic competitiveness. U.S.-China Economic and Security Review Commission.

Lanham, T., Chen, M., Radford, A., et al. (2023). Measuring faithfulness in chain-of-thought reasoning. arXiv preprint arXiv:2307.13702.

Lin, B., & Li, A. (2011). Impacts of carbon motivated border tax adjustments on competitiveness across regions in China. Energy, 36(8), 5111–5118. https://doi.org/10.1016/j.energy.2011.06.007

Manthorpe, J. (2014). China's rare earth monopoly: A strategic resource for the 21st century. Asian Affairs: An American Review, 41(3), 141–158. https://doi.org/10.1080/00927678.2014.932165

Meng, Y., Rong, Y., Zhang, Q., et al. (2023). Solo-instruct: Towards democratizing instruction data generation. arXiv preprint.

Ministry of Finance of China. (2007). Notice on adjusting the export tax rebate policy for certain commodities (Cai Shui [2007] No. 90). Ministry of Finance, State Administration of Taxation.

National Business Daily. (2012, April 20). Ganzhou xitu wuran diaocha: Daocai canliu 1.9 yi dun xitu feizha zhili xu 70 nian [Ganzhou rare earth pollution investigation: Illegal mining left 190 million tons of waste residue, remediation to take 70 years]. https://m.nbd.com.cn/articles/2012-04-20/648780.html

Nguyen, Q., Kermanshachi, S., & Hwang, E. (2013). Graph analytics for policy analysis: Using network metrics to evaluate policy networks. Proceedings of the 2013 IEEE International Conference on Intelligence and Security Informatics, 158–163.

OpenAI. (2025). Why language models hallucinate. OpenAI Research Blog. https://cdn.openai.com/pdf/d04913be-3f6f-4d2b-b283-ff432ef4aaa5/why-language-models-hallucinate.pdf

Petticrew, M., & Roberts, H. (2008). Systematic reviews in the social sciences: A practical guide. John Wiley & Sons.

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. Proceedings of EMNLP-IJCNLP 2019, 3982–3992.

Seaman, B. (2019). Rare earths: A strategic lever in international relations. Institut Français des Relations Internationales.

State Council of China. (2006). Comprehensive work plan for energy conservation and emission reduction during the 11th Five-Year Plan period (Guo Fa [2006] No. 28). State Council.

Timkey, W., & van Schijndel, M. (2021). All bark and no bite: Rogue dimensions in transformer language models obscure representational quality. Proceedings of EMNLP 2021, 2730–2746.

Turpin, M., Michael, J., Perez, E., & Bowman, S. R. (2024). Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. Advances in Neural Information Processing Systems (NeurIPS), 37.

van de Schoot, R., de Bruin, J., Schram, R., Zahedi, P., de Boer, J., Weijdema, F., ... & Kramer, B. (2021). An open source machine learning framework for efficient and transparent systematic reviews. Nature Machine Intelligence, 3(2), 125–133.

Wang, H., Jiang, X., & Xu, S. (2011). Analysis of export tax rebate policy of China based on computable general equilibrium model. Systems Engineering — Theory & Practice, 31(3), 434–444.

Wang, X., Wei, J., Schuurmans, D., et al. (2023). Self-consistency improves chain of thought reasoning in language models. International Conference on Learning Representations (ICLR).

Wei, J., Wang, X., Schuurmans, D., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. Advances in Neural Information Processing Systems (NeurIPS), 35.

World Trade Organization. (2014). China — Measures related to the exportation of rare earths, tungsten, and molybdenum: Reports of the Appellate Body (WT/DS431/AB/R, WT/DS432/AB/R, WT/DS433/AB/R). WTO.

Wübbeke, J. (2013). Rare earth elements in China: Policy and response strategies. Resources Policy, 38(4), 393–402. https://doi.org/10.1016/j.resourpol.2013.03.001

Xia, X., & Li, Y. (2024). Zhongguo xitu quan chanye lian zhong de huanjing he jiankang fengxian [Environmental and health risks across China's rare earth whole industry chain]. Xitu (Rare Earths), 2024(2). https://wap.cnki.net/touch/web/Journal/Article/XTZZ202402015.html

Xiao, S., Liu, Z., Zhang, P., & Mu, Y. (2023). C-Pack: Packaged resources to advance general Chinese embedding. arXiv preprint arXiv:2309.07597.

Zheng, G., & Wang, S. (2004). The impact of new export tax rebate policy on China's export trading forms: A new application of event-study analysis. Journal of Management Sciences in China, 7(3), 1–9.

Zheng, L., Chiang, W. L., Sheng, Y., et al. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. Advances in Neural Information Processing Systems (NeurIPS), 36.

Zhengyi Wang. (2011, March 30). Jiangxi Ganzhou xitu wuxu kaicai zhi shengtai yanzhong pohuai [Unregulated rare earth mining in Ganzhou, Jiangxi causes severe ecological damage]. Huashang Daily.